Analysis of Deep Learning Architectures for Real-Time Facial Recognition

Rahul Banyala (KU2407U434)
Prachi Ranjan (KU2407U433)
Rishu Pandey (KU2407U436)
Gauri Mahesh Adelkar (KU2407U401)

Abstract

This study deals with the real-time recognition of people using CNN-based deep learning models. It reviews and compares four different architectures: LightCNN-9, FaceNet, VGGFace, and MobileNetV1. The comparison uses the experimental results and figures reported in published research papers. Wherever possible, results obtained using the same facial dataset and evaluation protocol are selected to maintain a fair comparison. The models are compared based on recognition accuracy, computational requirements, inference speed, and suitability for real-time use. The study aims to identify which architecture provides the best balance between recognition performance and computational efficiency for a real-time facial recognition system. Later, the best-performing model will be used to build a real-time face recognition system using our own dataset.

I. Introduction

Facial recognition is a domain of computer vision that deals with recognising a human face and assigning a label or identity to it. It is commonly used in security systems, access control, attendance monitoring, surveillance, mobile-device authentication, and the organisation of digital images. Unlike face detection, which only finds the location of a face within an image, facial recognition attempts to determine the identity of the detected person. In a real-time facial recognition system, this complete process must be performed quickly enough to recognise people from a live image or video stream. Therefore, such a system requires not only good recognition accuracy but also efficient processing and a suitable response time.

The main challenge is that facial images can vary considerably, even when they belong to the same person. Lighting, camera angle, head position, facial expression, distance from the camera, image resolution, and partial obstruction can all affect the appearance of a face. Ageing and differences in camera quality add further variation. At the same time, different people may share some similar facial features. A recognition model must learn which patterns are useful for identifying a person and avoid relying too heavily on changes caused by the image or capture conditions. This is important in a live system, where images may not be as consistent as the carefully prepared images used in a research experiment.

Convolutional Neural Networks (CNNs) are widely used for image recognition because they learn features directly from image data. Early layers generally respond to simple patterns such as edges and textures, while deeper layers combine these into more complex shapes and facial features. CNNs reuse learned filters across an image, which helps them process spatial information without learning a separate weight for every pixel location. However, CNN architectures differ in how they learn identity, represent a face, and use computation. A model that is accurate on a benchmark may still be too large or slow for the hardware intended for a real-time application, so accuracy alone is not enough to choose a model.

This study reviews four architectures: LightCNN-9, FaceNet, VGGFace, and MobileNetV1. LightCNN was designed for face representation learning from large datasets that may contain noisy or incorrectly labelled images. Its Max-Feature-Map (MFM) operation compares pairs of feature maps and retains the stronger response, reducing the number of maps passed to later layers. LightCNN-9 is one of the smaller models in the study and produces a 256-dimensional face representation. These features make its balance of representation quality and computational cost relevant to our

project [1].

FaceNet takes a different approach to learning identity. It maps each face to a feature vector, or embedding, and trains the network with triplets consisting of an anchor image, a matching image, and a non-matching image. The training objective brings embeddings of the same person closer and separates those of different people. This representation can then be used for verification, recognition, or clustering, rather than relying only on a fixed output class for every person seen during training [2]. The result depends on the embedding model and training setup, so its reported scores need to be read together with the protocol used in the paper.

VGGFace uses a deep CNN for face recognition and studies how features learned from face images can be used to represent identity. The architecture follows a sequence of convolutional layers and pooling operations, followed by fully connected layers. The original work evaluates face recognition on benchmarks including LFW and YTF, and reports results under specific training and evaluation settings [3]. Its depth and feature representation provide a useful point of comparison with the more compact architectures, although the original paper does not report every efficiency measure needed for a direct real-time comparison.

MobileNetV1 was designed to reduce the computational cost of CNNs, particularly for mobile and resource-constrained devices. It uses depthwise separable convolutions: spatial filtering is performed separately for each input channel, and a pointwise convolution then combines information across channels. Width and resolution multipliers can further reduce the model's size and computation, with a corresponding trade-off in accuracy [4]. Since this original work focuses on general image classification, this study uses face-verification measurements reported for a MobileNetV1 baseline in a separate face-recognition study where available, rather than treating ImageNet results as facial-recognition results [5].

The four models therefore represent different priorities: feature selection and compact face representation in LightCNN-9, distance-based embeddings in FaceNet, a deep face-specific CNN in VGGFace, and efficient convolution in MobileNetV1. Comparing them helps explain why a single accuracy figure cannot determine which architecture is best for our application. The datasets, training methods, preprocessing, evaluation protocols, and hardware differ between the original studies. In particular, face verification asks whether two images show the same person, whereas our planned webcam application must assign a detected face to one of the people enrolled in our own dataset. Published verification accuracy is useful background, but it is not a direct measure of that final identification task.

For this reason, the comparison uses reported results and figures from the cited research papers, selecting similar datasets and metrics where possible. Differences in test conditions are noted rather than treated as if the measurements were directly equivalent. The architectures are considered in terms of recognition performance, embedding size, parameter count, computational requirements, and inference speed. Some sources do not report all of these values for the specific configuration considered, so missing values are identified instead of being estimated from another model variant. The final selection will require testing the candidate models on the same dataset and hardware, including the time needed for face detection, preprocessing, and recognition. The aim is to choose a model that recognises the required individuals reliably while remaining practical for real-time use.

II. Architecture 1: LightCNN-9

LightCNN was introduced by Wu et al. to learn facial features from large-scale datasets containing incorrectly labelled and noisy images [1]. The main objective of this architecture is to achieve reliable facial recognition while reducing the model size and computational requirements. LightCNN-9 is one of the smaller architectures proposed in the original study, making it suitable for systems where recognition speed and available hardware are important.

The main feature of LightCNN is the Max-Feature-Map (MFM) operation, which is

used instead of a conventional activation function such as ReLU. A convolutional layer first produces $2N$ feature maps, which are divided into pairs. MFM compares the corresponding values in each pair and keeps the maximum value:

$$\hat{x}_{ij}^{k} = \max\left(x_{ij}^{k}, x_{ij}^{k+N}\right) \tag{1}$$

As a result, $2N$ input feature maps are reduced to N output feature maps. This creates competition between the feature maps and allows the network to retain the stronger facial information while suppressing weaker or noisy responses.

LightCNN-9 takes a 128×128 grayscale facial image as its input. The first stage contains a 5×5 convolution followed by MFM and max pooling. It is followed by four convolution blocks, with each block containing a 1×1 convolution, MFM, a 3×3 convolution, and another MFM operation. Four max-pooling operations gradually reduce the spatial dimensions to $8 \times 8 \times 128$. The resulting features are flattened into 8,192 values and passed through a fully connected layer containing 512 outputs. MFM then reduces this output to a compact 256-dimensional facial representation. Finally, a fully connected classifier produces a score for each identity class. The complete flow of the architecture is presented in Figure 1.

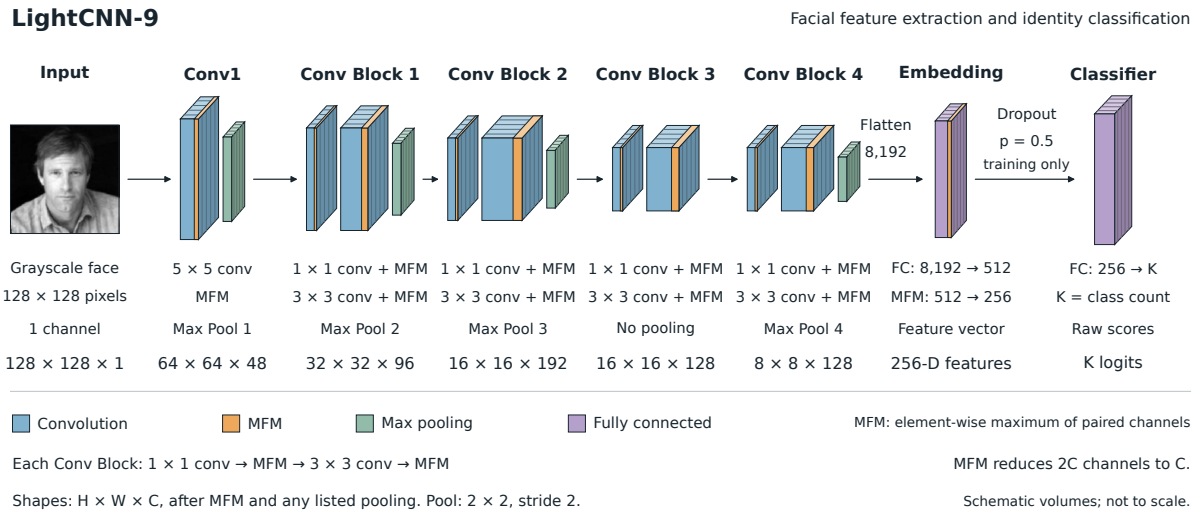


Figure 1: *Architecture of LightCNN-9 showing the convolution blocks, MFM operations, pooling layers, 256-dimensional embedding, and identity classifier.*

According to the original paper, LightCNN-9 contains approximately 5.556 million parameters and requires approximately 1 GFLOP. It produces a 256-dimensional facial representation and requires approximately 67 ms for feature extraction on a single core of an Intel i7-4790 processor. The paper reports 98.80% accuracy on LFW and 93.40% accuracy on YTF under its stated evaluation protocols [1].

The smaller model size, low-dimensional representation, and use of grayscale images make LightCNN-9 a suitable candidate for real-time facial recognition. However, its fully connected layer requires a fixed input size, and the reported processing time depends on the hardware used by the original authors. The LFW result also represents face verification rather than the identity-classification task required by our final system. For our implementation, the final classifier can be replaced with a new classifier for the identities in our own dataset while retaining the earlier layers for facial feature extraction.

III. Architecture 2: FaceNet

FaceNet is a unified system for face verification, recognition, and clustering, which uses a deep convolutional network trained with triplet loss. Each face is mapped onto the Euclidean space. The net-

work is trained such that the embeddings of similar data points converge closer together within the feature space, while simultaneously pushing embeddings of dissimilar data points to drift further apart; meaning images of the same person are close together in the learned space and vice versa in the case of images of different people [2].

It also uses a triplet loss function. To calculate triplet loss, we use images corresponding to the anchor, positive, and negative.

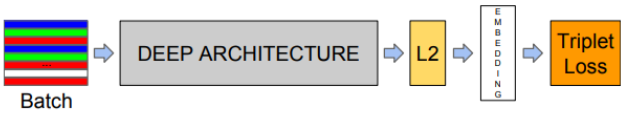


Figure 2: *Model Structure of FaceNet*

The network consists of an input layer, a deep CNN, followed by L2 normalization, which results in the face embedding. This is then followed by the triplet loss.

Triplet Loss and Selection

The loss function is defined using triplets of training points of the form (A, P, N).

A = anchor (reference image)
P = positive (if the image is of the same person as A)
N = negative (if the image is of a different person than A)

Triplet loss minimizes the distance between the anchor and the positive, while maximizing the distance between the anchor and the negative (see Figure 3).

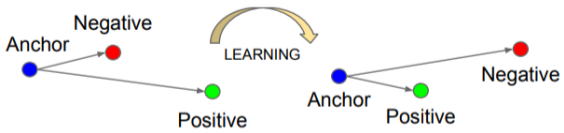


Figure 3: *Triplet Loss*

Triplet loss can be formalized as:

$$L = \sum_{i}^{N} \Big[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha \Big]_+ \tag{2}$$

α here is defined as the margin that is enforced between positive and negative pairs.

Generating all possible triplets would result in multiple triplets that already satisfy the constraint. These triplets would not contribute to the training and would result in slower convergence. And thus it is important to select hard triplets which can therefore contribute to improving the model.

Deep Convolution Networks

FaceNet trains CNNs using Stochastic Gradient Descent (SGD) with standard backpropagation and AdaGrad.

SGD is an optimization algorithm used to reduce the loss function.

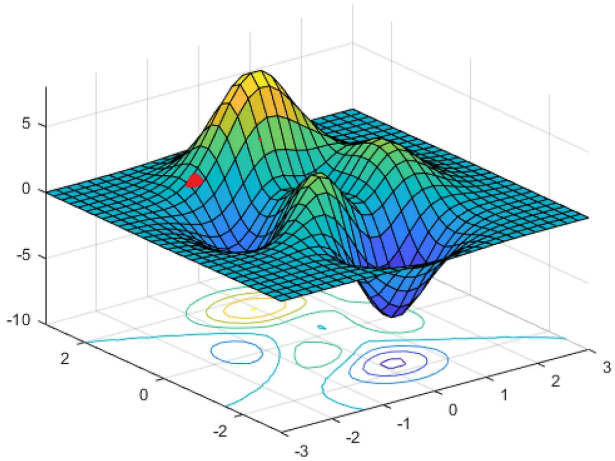


Figure 4: *SGD (Credit - datascience.stackexchange)*

The x-axis and y-axis represent the weights, while the z-axis is the corresponding loss. SGD updates the weights to move towards lower loss, although it does not guarantee reaching a global minimum.

In SGD, the gradient is a vector which indicates the direction of the steepest local increase. SGD then updates the parameter in the opposite direction to minimize the loss, using gradients estimated from randomly sampled training data.

AdaGrad adapts the learning rate for each parameter using its accumulated squared gradients.

ReLU (Rectified Linear Unit)

ReLU is a non-linear activation function. Without non-linear activation functions, even multiple layers of convolution would collapse into a single linear transformation. Also, without activation functions, complex and intricate patterns cannot be learned effectively, as the decision boundary would remain linear.

ReLU is a widely used alternative to traditional activation functions such as sigmoid and tanh. This activation function helps mitigate the vanishing gradient problem, as its gradient remains constant for positive inputs.

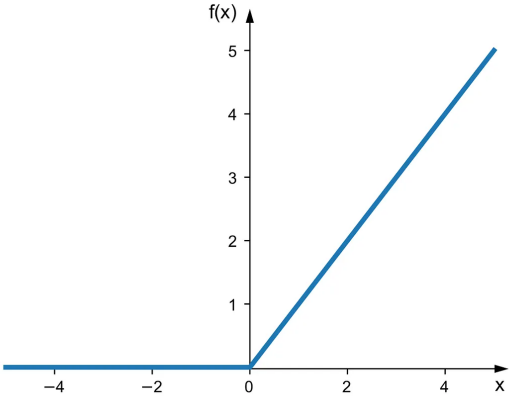


Figure 5: *ReLU activation function*

CNN Architectures

The two types of CNN architectures used in FaceNet are Zeiler & Fergus architecture and Inception Model.

Zeiler & Fergus architecture - This architecture introduced a novel visualization technique that gives insight into the function of intermediate layers and the operation of classifiers in a CNN.

layer	size-in	size-out	kernel	param	FLPS
conv1	220×220×3	110×110×64	7×7×3, 2	9K	115M
pool1	110×110×64	55×55×64	3×3×64, 2	0	
rnorm1	55×55×64	55×55×64		0	
conv2a	55×55×64	55×55×64	1×1×64, 1	4K	13M
conv2	55×55×64	55×55×192	3×3×64, 1	111K	335M
rnorm2	55×55×192	55×55×192		0	
pool2	55×55×192	28×28×192	3×3×192, 2	0	
conv3a	28×28×192	28×28×192	1×1×192, 1	37K	29M
conv3	28×28×192	28×28×384	3×3×192, 1	664K	521M
pool3	28×28×384	14×14×384	3×3×384, 2	0	
conv4a	14×14×384	14×14×384	1×1×384, 1	148K	29M
conv4	14×14×384	14×14×256	3×3×384, 1	885K	173M
conv5a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv5	14×14×256	14×14×256	3×3×256, 1	590K	116M
conv6a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv6	14×14×256	14×14×256	3×3×256, 1	590K	116M
pool4	14×14×256	7×7×256	3×3×256, 2	0	
concat	7×7×256	7×7×256		0	
fc1	7×7×256	1×32×128	maxout p=2	103M	103M
fc2	1×32×128	1×32×128	maxout p=2	34M	34M
fc7128	1×32×128	1×1×128		524K	0.5M
L2	1×1×128	1×1×128		0	
total				140M	1.6B

Figure 6: *Zeiler & Fergus architecture used in FaceNet*

Inception Model - The architecture was first introduced by the researchers at Google to curb the exorbitant resource requirement of deep neural networks. The Inception architecture combines 1×1, 3×3 and 5×5 convolution filters in parallel and extracts features from the input image at different scales. The output of each filter is concatenated and passed to the next layer. This process is then repeated. This approach helps capture the local and global features while also minimizing the number of parameters required.

type	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj (p)	params	FLOPS
conv1 (7×7×3, 2)	112×112×64	1							9K	119M
max pool + norm	56×56×64	0						m 3×3, 2		
inception (2)	56×56×192	2		64	192				115K	360M
norm + max pool	28×28×192	0						m 3×3, 2		
inception (3a)	28×28×256	2	64	96	128	16	32	m, 32p	164K	128M
inception (3b)	28×28×320	2	64	96	128	32	64	L_2, 64p	228K	179M
inception (3c)	14×14×640	2	0	128	256,2	32	64,2	m 3×3,2	398K	108M
inception (4a)	14×14×640	2	256	96	192	32	64	L_2, 128p	545K	107M
inception (4b)	14×14×640	2	224	112	224	32	64	L_2, 128p	595K	117M
inception (4c)	14×14×640	2	192	128	256	32	64	L_2, 128p	654K	128M
inception (4d)	14×14×640	2	160	144	288	32	64	L_2, 128p	722K	142M
inception (4e)	7×7×1024	2	0	160	256,2	64	128,2	m 3×3,2	717K	56M
inception (5a)	7×7×1024	2	384	192	384	48	128	L_2, 128p	1.6M	78M
inception (5b)	7×7×1024	2	384	192	384	48	128	m, 128p	1.6M	78M
avg pool	1×1×1024	0								
fully conn	1×1×128	1							131K	0.1M
L2 normalization	1×1×128	0								
total									7.5M	1.6B

Figure 7: *Inception Model used in FaceNet*

Performance of FaceNet Model on LFW

The model is evaluated on two modes:

Fixed center crop of the LFW provided thumbnail.

A face detector is run on the LFW thumbnails. If it fails to align the images, the LFW alignment is used.

FaceNet achieves a face-verification accuracy of **98.87%** $\pm$ **0.15** with the fixed center crop and **99.63%** $\pm$ **0.09** with extra face alignment; the uncertainties are standard errors of the mean [2].



Figure 8: *LFW errors. This shows all pairs of images that were incorrectly classified on LFW*

Conclusion

With a verification accuracy of **99.63% ± 0.09** on LFW, FaceNet demonstrates the effectiveness of embedding-based approaches. This result is reported for the NN1 model, which has approximately 140 million parameters; the 7.5-million-parameter NN2 is a different Inception configuration [2].

IV. Architecture 3: VGGFace

Overview and design objective

VGGFace is a deep convolutional network for face recognition, which was introduced in the paper "Deep Face Recognition". Earlier, the best face recognition systems belonged to giant companies like Google, which trained on about 200 million face images, that nobody else could access. The Oxford team wanted to show that you don't need a giant private dataset for efficient facial recognition. So, the first thing they did was to build a huge public face dataset cheaply by using web searches and automatic filtering with only a little human checking. The result was 2.6 million images of 2,622 people. Second, they trained a simple but very deep CNN on it.

Main architectural feature

VGGFace has no new special layer. It differs from an ordinary CNN as it combines a very deep but plain architecture with a two-stage training. It stacks many small 3×3 convolutions with ReLU and max-pooling, and it doesn't add the extra LRN step that AlexNet had. That is why it is described as "simple". In stage 1, the network is trained as a 2,622-class classifier, which makes training easier and faster. In stage 2, only a final projection layer is trained with triplet loss, which pulls faces of the same person closer and pushes different people apart.

$$E(W') = \sum_{(a,p,n)\in T} \max\{0, \alpha - \|\mathbf{x}_a - \mathbf{x}_n\|_2^2 + \|\mathbf{x}_a - \mathbf{x}_p\|_2^2\}, \quad \mathbf{x}_i = W' \frac{\phi(\ell_i)}{\|\phi(\ell_i)\|_2}$$

Architecture flow

As the figure shows, a 224×224 face image, with the average training face subtracted, passes through stacks of 3×3 convolutions with ReLU, separated by max-pooling layers, while the number of filters grows from 64 to 512. Three fully connected layers follow: FC6 and FC7 with 4,096 outputs, then

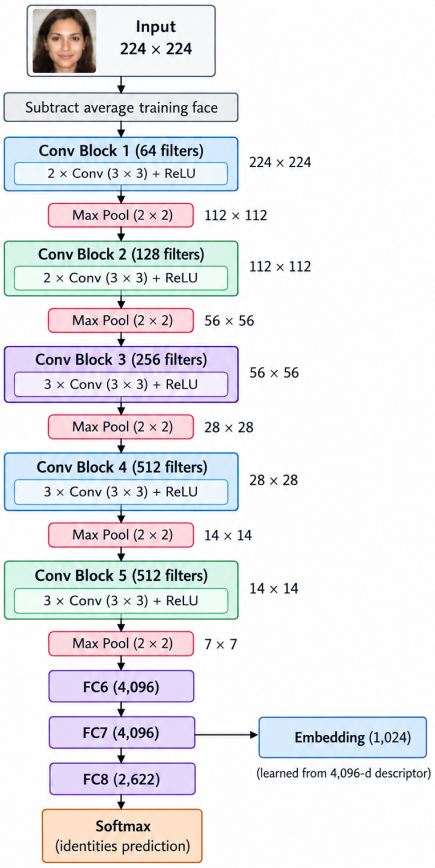


Figure 9: *VGGFace architecture (adapted from Parkhi et al. [3]).*

- **Input:** 224×224 face image (after subtracting average training face)
- **Convolution blocks:** 3×3 conv + ReLU, with max-pooling (filters: $64 \to 128 \to 256 \to 512 \to 512$)
- **Fully connected layers:** FC6 (4,096), FC7 (4,096), FC8 (2,622) $\to$ Softmax
- **Embedding:** 1,024-d vector learned from the 4,096-d descriptor (from FC7)

FC8 with 2,622 outputs for the identities and a softmax. A 1,024-d embedding is later learned from the 4,096-d descriptor.

Training and evaluation setup

The network was first trained as an identity classifier with softmax loss on the authors' dataset of 2.6 million images of 2,622 people, and a final projection was then learned with a triplet loss for the embedding. Training images were rescaled to 256 pixels on the shorter side, randomly cropped to 224×224 and flipped with 50% probability. Testing used LFW and YouTube Faces, whose identities were removed from the dataset, under the unrestricted protocol. Only face verification was evaluated, classification accuracy was not reported.

Reported performance and efficiency

Under the unrestricted protocol, the authors report 98.95% accuracy on LFW, and 99.13% in 100%-EER terms with embedding learning [3]. On YouTube Faces, accuracy rises from 91.6% to 97.3% with embedding learning. A single network trained on 2.6 million images matched the state of the art closely on LFW, using far less data than FaceNet's 200 million. Training used four Titan Black GPUs (6 GB each); parameter count, FLOPs and inference time were not reported.

Strengths, limitations and suitability

Its main strength is simplicity: a plain deep CNN reaches near state-of-the-art verification results using 2.6M training images, far fewer than FaceNet's 200M [3]. However, the authors needed four GPUs because of the large memory footprint, and test features were averaged over 10 crops at each of three scales. As speed and model size are not reported, we expect it to be heavy for real-time webcam use; this needs testing.

V. Architecture 4: MobileNetV1

Introduction

MobileNet is a lightweight architecture developed for mobile and embedded devices, where memory and processing power are limited. It aims to keep computation low while maintaining useful accuracy [4]. This makes it a suitable candidate for our facial recognition system, where we need to consider both recognition accuracy and the time taken to process a face.

Depthwise separable convolution

In a standard convolution, filtering features and combining information from different channels happen together. MobileNet splits this into two operations. Depthwise convolution applies one filter to each input channel. Pointwise convolution then uses 1×1 filters to combine these outputs and produce new features [4]. Figure 10 shows the difference between the two blocks.

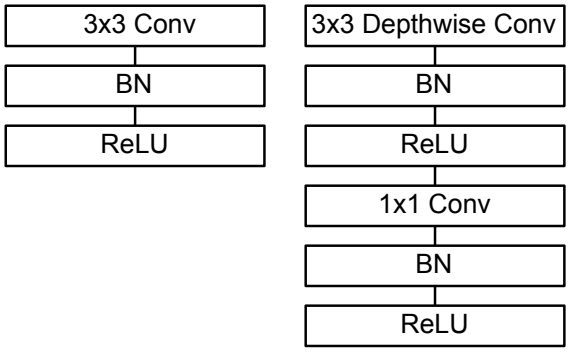


Figure 10: *Standard and depthwise separable convolution. BN denotes batch normalization. Reproduced from Howard et al. [4].*

For example, consider an input with three channels and a layer that needs 64 output channels. A standard 3×3 convolution requires 1,728 weights. Splitting it into depthwise and pointwise operations reduces this to 219 weights, excluding biases and batch normalization. The output can have the same dimensions while requiring much less computation.

Computational cost and parameters

Let D_K be the kernel size, D_F the feature-map size, and M and N the input and output channel counts. Assuming equal input and output spatial dimensions, the computation costs are [4]:

$$C_{\text{standard}} = D_K^2 M N D_F^2 \tag{3}$$

$$C_{\text{separable}} = D_K^2 M D_F^2 + M N D_F^2 \tag{4}$$

For 3×3 kernels, the paper reports roughly eight to nine times less computation. The baseline ImageNet model contains 4.2 million parameters and requires 569 million multiply-adds, achieving 70.6% accuracy [4]. These figures describe ImageNet classification, not face verification.

Core architecture and GEMM

MobileNet begins with a standard convolution, followed by depthwise and pointwise convolution blocks. Each convolution is followed by batch normalization and ReLU. Average pooling reduces the final feature map, and a fully connected layer followed by softmax produces class probabilities. Counting depthwise and pointwise convolutions separately, the network has 28 layers [4].

Most computation occurs in the 1×1 pointwise convolutions. These can directly use general matrix

multiplication, or GEMM, without the memory rearrangement often needed for standard convolutions. MobileNet can therefore carry out most of its work using optimized matrix-multiplication routines [4].

Width and resolution multipliers

The width multiplier reduces the number of channels throughout the network. For example, a multiplier of 0.5 approximately halves the channels, reducing both parameters and computation. Making the network smaller can also reduce accuracy.

The resolution multiplier reduces the input image size and the spatial size of its feature maps. This lowers computation while keeping the parameter count unchanged at a fixed width. Together, these settings allow us to adjust the model to the available resources [4]. Figures 11 and 12 show these trade-offs on ImageNet.

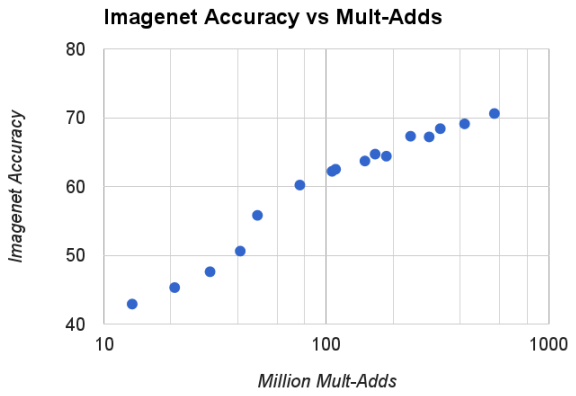


Figure 11: *ImageNet accuracy and computation for different MobileNet widths and input resolutions. Reproduced from Howard et al. [4].*

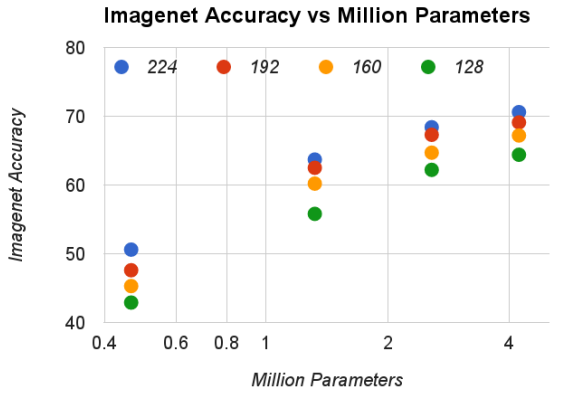


Figure 12: *ImageNet accuracy and parameter count. Colours indicate input resolution; changing resolution alone does not change the parameter count. Reproduced from Howard et al. [4].*

Face verification performance

The original MobileNet paper studies face embeddings but does not report LFW accuracy. For this comparison, we use the MobileNetV1 baseline evaluated by Chen et al. in the MobileFaceNets paper [5].

Their baseline was trained on CASIA-WebFace using ArcFace loss and achieves 98.63% accuracy on LFW, with 3.2 million parameters. Inference takes 60 ms on a Snapdragon 820 CPU using four threads and NCNN [5]. These results belong to the adapted MobileNetV1 baseline, not MobileFaceNet. YTF accuracy and the baseline's embedding dimension are not explicitly reported.

Conclusion

MobileNetV1 reduces computation through depthwise separable convolutions, while its width and resolution can be adjusted when resources are limited. Its reported accuracy and speed still depend on the training setup and hardware. We therefore need to test it on our own dataset and measure the complete processing time before selecting it for real-time facial recognition.

VI. Analysis and Discussion

The four architectures are compared using published research results. LFW provides the primary face-verification benchmark, while YTF evaluates face pairs from videos with changes in pose, expression, lighting and image quality. MobileNetV1's face-verification results come from Chen et al. [5], rather than its original ImageNet experiment. The final comparison in Table 1 records accuracy, parameter count, embedding dimension and inference time, with missing values and testing conditions explained alongside it.

These results help identify candidates, but differences in training, evaluation and hardware prevent a direct overall ranking. Our final selection requires tests on the same dataset and hardware, including the time needed for face detection, preprocessing and recognition.

VII. Conclusion

This study compares LightCNN-9, FaceNet, VGGFace and MobileNetV1 for a real-time facial recognition system. Each architecture uses a different

approach to learning facial features and managing computational cost. Published results help identify suitable candidates, but differences in training data, evaluation protocols and hardware prevent a direct ranking. Face verification also differs from identifying people in our webcam system. Several values are not explicitly reported in the selected sources: NN1 inference time, VGGFace's parameter count and inference time, and MobileNetV1's YTF accuracy and embedding dimension. NR means not reported; these values are not zero or estimates from other model variants. The papers do not explain every omission. Our final choice will require testing accuracy, model size and complete processing time on our own dataset and hardware.

Table 1: *Final dataset and efficiency comparison using published research results. NR denotes a value not explicitly reported for the selected configuration.*

Architecture	LFW Acc. (%)	YTF Acc. (%)	Parameters	Embedding	Inference time
LightCNN-9	98.80	93.40	5.556M	256-D	67 ms[a]
FaceNet[b]	99.63	95.12	140M	128-D	NR
VGGFace[c]	98.95	97.3	NR	1,024-D	NR
MobileNetV1[d]	98.63	NR	3.2M	NR	60 ms[e]

[a] Feature extraction on one Intel i7-4790 core; the LFW setting is unsupervised, unlike the unrestricted FaceNet and VGGFace settings. Source: Wu et al., Tables IV and X [1].

[b] LFW: $99.63\% \pm 0.09$ with extra alignment for NN1 (about 140M parameters); NN2 has 7.5M parameters and is not the model attached to this score. YTF: $95.12\% \pm 0.39$ using 100 detected frames per video; the paper does not explicitly identify the backbone in that subsection. Source: Schroff et al., Table 1 and Sections 5.6–5.7 [2].

[c] LFW accuracy is 98.95%; 99.13% is $100\% -$ EER, a different metric. YTF accuracy is 97.3% with embedding learning and $K = 100$; $100\% -$ EER is 97.4%. Later efficiency values for a 4,096-dimensional VGG descriptor are not substituted for this 1,024-dimensional configuration. Source: Parkhi et al., Tables 4–6 [3]; Wu et al. [1].

[d] MobileNetV1 baseline trained on CASIA-WebFace with ArcFace loss, not MobileFaceNet or the original ImageNet classifier. Its 88.95% AgeDB-30 result is not a YTF result. Source: Chen et al., Table 2 [5].

[e] Snapdragon 820 CPU, four threads, NCNN; this timing is not directly comparable with LightCNN-9.

References

[1] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 815–823, 2015.

[3] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep face recognition," in *Proceedings of the British Machine Vision Conference*, 2015.

[4] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861v1*, 2017.

[5] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for accurate real-time face verification on mobile devices," *arXiv preprint arXiv:1804.07573v4*, 2018.